

DevOps Engineer Technical Assignment

AWS Free Tier – Production-like Web Application

Candidate: [Your Name] **Date:** [Submission Date] **GitHub Repository:** [Paste Link]

Important: Replace all bracketed placeholders and add your own screenshots before submission.

1. Objective

This project deploys a simple Node.js web application/API on an Amazon EC2 instance. The solution uses Amazon S3 for backups/static assets, IAM least-privilege access, Security Groups, GitHub Actions CI/CD, Amazon CloudWatch monitoring and k6 load testing.

2. Architecture

Flow: User → Internet → EC2 Security Group (80/443) → Nginx → Node.js application managed by PM2. GitHub → GitHub Actions → SSH deployment to EC2. EC2 metrics/logs → CloudWatch. Application backup/static assets → private S3 bucket.

Insert architecture diagram screenshot here: [ARCHITECTURE DIAGRAM]

3. Infrastructure Setup

- Created an Ubuntu EC2 Free Tier instance: [instance ID / region].
- Configured Security Group: SSH 22 restricted to my public IP; HTTP 80 open to internet; HTTPS 443 open only after SSL setup.
- Installed Nginx, Node.js, Git and PM2. Nginx reverse-proxies traffic to Node.js port 3000.
- Created private S3 bucket: [bucket name]. Block Public Access enabled. Used for application backup/static assets.
- Created IAM role/user with only required permissions. Root credentials were not used for deployment.

Evidence: [EC2 screenshot] [Security Group screenshot] [S3 screenshot] [IAM policy screenshot]

4. Application Deployment

The sample application exposes `/` for the web page and `/health` for health status. The application is started using **pm2 start app.js --name devops-app**. Nginx configuration forwards port 80 requests to localhost:3000. Validation URL: **http://[EC2-PUBLIC-IP]/health**.

Deployment commands used:

```
sudo apt update && sudo apt install -y nginx git nodejs npm
sudo npm install -g pm2
git clone [REPOSITORY_URL] app && cd app
npm install
pm2 start app.js --name devops-app
pm2 save
```

5. CI/CD Pipeline – GitHub Actions

On every push to the main branch, the pipeline checks out code, installs dependencies, runs tests/build, securely connects to EC2 using GitHub Secrets, pulls the latest code, installs packages and restarts PM2. Secrets used: `EC2_HOST`, `EC2_USER` and `EC2_SSH_KEY`. No secrets are stored in the repository.

Pipeline result: [Insert successful GitHub Actions screenshot]

6. Security Summary

- Least privilege IAM policy applied; no daily use of root account.
- SSH restricted to my IP address; application ports separated from internal Node.js port.
- S3 bucket is private and public access is blocked.
- Secrets stored in GitHub Actions Secrets, not in source code.
- Nginx and OS packages updated; application runs under PM2.

HTTPS: [Configured with domain and Let's Encrypt / Not configured because a domain was unavailable; HTTP demonstrated].

7. Monitoring and Logs

CloudWatch EC2 metrics were enabled. A CPU utilization alarm was configured for CPU > 70% for 5 minutes, with notification target [email/SNS]. Dashboard includes CPU utilization, network traffic and status checks. Application logs reviewed: /var/log/nginx/access.log, /var/log/nginx/error.log and PM2 logs.

Evidence: [CloudWatch dashboard screenshot] [Alarm screenshot] [Logs screenshot]

8. Load Testing (k6)

Test command: **k6 run load-test.js**. Test scenario: [e.g., 10 virtual users for 30 seconds].

Metric	Observed Result
Virtual users	[fill]
Requests / throughput	[fill]
Average response time	[fill]
p95 response time	[fill]
Error rate	[fill]
EC2 CPU / memory	[fill]

Observation and optimization: During higher traffic, response time/CPU may increase. Recommended improvements are autoscaling, Application Load Balancer, caching, CloudFront for static assets, database optimization and moving secrets to AWS Secrets Manager.

9. Deliverables Checklist

- GitHub repository link: [link]
- Deployment guide: this document / README
- Architecture diagram: [attached]
- Pipeline configuration: .github/workflows/deploy.yml
- Monitoring screenshots: [attached]
- Load-testing report/graphs: [attached]
- Security summary: Section 6
- 5–10 minute demo video link: [link]

10. Conclusion

The application was designed as a small production-like AWS deployment with automated delivery, access control, monitoring, logs and performance validation. Future improvements include HTTPS with a custom domain, load

balancer/autoscaling and centralized log retention.